



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

تقرير انجاز مشروع البرمجة التفرعية

احمد يوسف خميس

إسماعيل سعيد الوغا

محمد خالد سليمان خشفه

سيدرا محمد قاسم

عبد الرحمن حسان عبدو

بإشراف المهندس :

حذيفة عقيل

(ملاحظة الصور و ملفات الاختبار موجودة في مجلد يحتوي مجلد لكل طلب فيه صور او ملفات الخرج)

الطلب الأول : حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

لإثبات وجود مشكلة التزامن وتأثيرها المباشر على سلامة البيانات، قمنا بإنشاء مسار تجريبي غير آمن (Unsafe Experimental Path) مؤقتاً داخل المتحكم `TestCheckoutController.php` لمحاكاة السلوك الافتراضي للنظام قبل معالجته، وسيتم إزالة هذا المسار بعد التوثيق. وايضا حقل المخزون (stock) في قاعدة البيانات معرّف بنوع unsigned، مما يمنع قيمته ان تكون سالبة فقط يصبح صفر؛ ومع ذلك، فإن المشكلة الأساسية لم تكن في القيمة السالبة، بل في توليد أوامر شراء ناجحة (Orders) تتجاوز العدد الفعلي المتاح في المخزون، مثل إنشاء عدة طلبات لمنتج لا تتوفر منه سوى قطعة واحدة.

ولحل هذه المشكلة ، تم اعتماد أسلوب العمليات الذرية المشروطة (Conditional Atomic Decrement) المدعومة على مستوى قاعدة البيانات ومغلقة داخل عملية مركبة (Database Transaction) يضمن هذا الحل دمج عمليتي التحقق من الشرط والخصم الفعلي في استعلام (Query) واحد غير قابل للانقسام لمنع التداخل بين الطلبات المتزامنة، يتم التنفيذ برمجياً كالتالي:

```
DB::transaction(function () use ($productId, $qty) {  
    // الخصم الذري المشروط يمنع أي تداخل بين الطلبات المتزامنة  
    $updated = Product::where('id', $productId)  
        →where('stock', '≥', $qty)  
        →decrement('stock', $qty);  
  
    if (! $updated) {  
        throw new Exception('Out of stock or insufficient  
inventory.');    }  
});
```

لماذا اخترنا هذا الحل:

لأنه يضمن عدم حدوث تعارض، وكفاءته عالية؛ وهو أفضل من القفل المتشائم إذا كانت الحالة بسيطة بدون كوبونات أو ما شابه، حيث يكفي بقفل السطر ويضمن أداءً عالياً.

نتائج الاختبار:

تم استخدام سكربت محاكاة التزامن المكثف `concurrent_checkout_test.php` عبر أداة JMeter

بـ 5 Threads متزامنة في نفس الأجزاء من الثانية، وأظهرت النتائج المخيرية المصورة ما يلي:

- **في المسار غير المحمي (قبل الحل):** عند اختبار 5 حالات شراء متزامنة لمنتج ذي مخزون محدود (قطعة واحدة)؛ كانت النتيجة نجاح الحالات الخمس بالكامل وتوليد 5 أوامر شراء (Overselling) على الرغم من عدم كفاية المخزون المادي.
- **في المسار المحمي (بعد الحل):** عند إعادة تشغيل نفس السيناريو، فشلت 4 عمليات وتم رفضها بأمان، ولم تنجح سوى عملية واحدة فقط (1) وهي التي التقطت قفل السطر أولاً واستحقت المخزون المتاح.

الطلب الثاني: إدارة الموارد الحاسوبية (Resource Management & Capacity Control)

كيف تم الحل:

تم تنفيذ آلية Rate Limiting ديناميكية داخل `RouteServiceProvider.php`. بدلاً من الاعتماد على قيم ثابتة، يتم احتساب الحدود برمجياً استناداً إلى إعدادات السيرفر الممررة عبر ملف `env`. مثل عدد الـ `Threads` في `Apache` المستضاف محلياً على `XAMPP`، ومتوسط زمن الاستجابة، ونسبة الاستهلاك المستهدفة. تم توزيع هذه القدرة الاستيعابية على مجموعات حماية مخصصة: `auth`، `cart`، `checkout`، و `products`.

لماذا اخترنا هذا الحل:

لأنه يربط قيود الطلبات بقدرة السيرفر الفعلية، ويسهل تعديل الإعدادات من ملف الانفايرومنت دون تغيير الكود، وهو بيعطيني مرونة عند تغيير حجم السيرفر.

كيف تمت عملية التحقق:

(ملاحظة: في البداية كنت قد وضعت الـ `ریت` لـ `متنغ ستاتك` ولكن عدلته عند حل الطلب الرابع).

تمت مقارنة سلوك النظام تحت ضغط اختبار حمل مكثف (Load Test) يصل إلى 230 مستخدم وهمي متزامن (Max VUs) عبر أداة `k6`:

- **قبل تطبيق الحل (بدون قيود):** تكدست الطلبات ووصل متوسط زمن الاستجابة إلى 19.17 ثانية، وبلغت ذروة التأخير في الطابور (p95) حوالي 31.87 ثانية. تسبب هذا الضغط غير المحدود في استهلاك مفرط للموارد؛ حيث التهم النظام ذاكرة عشوائية قاسية بلغت ذروتها (Max RAM) حوالي 2480.61 MB مع متوسط استهلاك معالج بـ 16.68% (وذروة وصلت إلى 31.77%).
- **بعد تطبيق الحل الديناميكي:** عند تشغيل نفس الاختبار، نجح النظام في صد الضغط وحماية الموارد عن طريق تفعيل الـ `Too Many Requests 429`؛ حيث رصد الاختبار خروج 44 طلب مقيد للـ `auth`، و 34 للـ `cart`، و 28 للـ `checkout`.
- **النتيجة على الموارد:** انخفض متوسط استهلاك المعالج (Avg CPU) بشكل ملحوظ ليصل إلى 5.71% (بذروة 25.34%)، وهبط استهلاك الذاكرة العشوائية الأقصى إلى 1881.07 MB، مما يثبت نجاح الـ `Throttling` في حماية الخادم من الانهيار وتوفير عزل كامل للموارد.

الطلب الثالث: المعالجة غير المتزامنة (Asynchronous Queues)

كيف تم الحل:

تم الانتقال بالكامل من المحرك المتزامن (sync) إلى محرك الطوابير المعتمد على قاعدة البيانات (database driver) لتعمل المهام الثقيلة في الخلفية. تم تحويل العمليات المستهلكة للوقت إلى ثلاث وظائف خلفية مستقلة (Background Jobs):

- **ProcessOrderJob**: لمعالجة الدفع وتحديثات حالات الطلب بعد ال-checkout.
 - **GenerateOrderSummaryJob**: لتوليد ملفات ملخصات الطلبات للمشتريين.
 - **GenerateDailySalesReportJob**: لتجميع التقارير البيعية اليومية الضخمة.
- تم إضافة جدول jobs عبر migration مخصص، وضبط منطق تراجع المحاولات المتدرج تلقائياً عند الفشل (retry/backoff).

لماذا اخترنا هذا الحل:

- لا يعتمد على تشغيل خدمات خارجية إضافية مما يجعله مناسباً لحجم المشروع الحالي.
- يسمح بمراقبة ومتابعة المهام المخزنة بروية مباشرة داخل جداول قاعدة البيانات.

كيف تم التحقق (الأدلة الرقمية من اختبارات الأداء):

تم الفحص بواسطة سيناريو تحميل معتدل متوازن (env LOAD_PROFILE=moderate--) عبر أداة k6 للتأكد من انتقال المعالجة إلى الخلفية ومراقبة الموارد عبر [project_resource_monitor.php](#) الممتد على مدار 90 ثانية:

- **تحسن أوقات الاستجابة**: انخفض زمن الاستجابة العام لـ 95p بشكل حاد ليصل إلى 12.12 ثانية فقط (حيث سجلت مسارات المنتجات والسلة حوالي 10.79 ثانية)، ولم يعد العميل ينتظر انتهاء المعالجة على خادم الويب مباشرة.
- **كفاءة استهلاك الموارد**: أظهر مراقب الموارد ثباتاً ممتازاً؛ حيث استقر متوسط استهلاك المعالج (Avg CPU) عند 3.26% فقط (بذروة ترحيل مؤقتة بلغت 52.58%)، وحافظت الذاكرة على استقرار ممتاز بمتوسط استهلاك 1423.31 MB.

الطلب الرابع: معالجة البيانات الضخمة على دفعات (Batch Processing)

كيف تم الحل:

تم تنفيذ المعالجة الدفعية داخل [GenerateDailySalesReportJob.php](#) باستخدام خاصية تقطيع البيانات (chunk(100 حتى تُعالج سجلات الطلبات على دفعات صغيرة بدلاً من تحميل كل السجلات الضخمة دفعة واحدة في الذاكرة. ولإظهار الأثر عملياً تمت إضافة:

- كلاس [ResourceMonitor.php](#) لمراقبة الأداء محلياً.
- سكربت [batch_processing_demo.php](#) لتوليد بيانات ضخمة وتشغيل التقرير.
- سجلات (Logs) واضحة تظهر تقسيم العمليات بصيغة Chunk #1, Chunk #2.

لماذا اخترنا هذا الحل:

يضمن تقليل استهلاك الذاكرة العشوائية للحد الأدنى، ويمنع حدوث خطأ نفاذ الذاكرة الشهير (Fatal Error: Allowed Memory Size Exhausted) عند نمو البيانات، ويحقق الهدف الأكاديمي بوضوح في سجلات النظام.

كيف تم التحقق:

أظهرت سجلات النظام (laravel.log) تقسيم البيانات بنجاح وثبات كامل للموارد:

Processing Sales Report Batch - Chunk #1 | Orders in this batch: 100

Processing Sales Report Batch - Chunk #2 | Orders in this batch: 100

...

الطلب الخامس: توزيع الأحمال (Load Distribution)

كيف تم الحل:

لتوضيح كيفية تشغيل التطبيق على أكثر من خادم وكيفية توزيع الأحمال بينها، قمنا بتجهيز بيئة اختبار تحاكي خوادم الإنتاج وفق المعايير التالية:

- البنية: استخدمنا موازن الأحمال في أباتشي mod_proxy_balancer كـ Load Balancer أمامي يقود إلى مجموعتين من خوادم التطبيق الخلفية تعمل على منافذ مختلفة.
- إثبات الاتصال: أضفنا نهاية مسار اختبارية /server-info تعيد هوية الخادم الحالي الذي استقبل الطلب (PID).
- الاستراتيجية المتبعة: تم تطبيق سياسة التوزيع الدائري Round-Robin مع فحوصات الصحة (Health Checks) لتبسيط العرض الأكاديمي.

كيف تم التحقق:

عند تشغيل سكريبت k6_load_balance_demo.js ليصوب طلبات متتالية إلى نقطة التحميل الأمامية، أظهرت النتائج توزيعاً متقارباً ومتوازناً للطلبات بين الخوادم الخلفية بنسب (52% مقابل 48%)، مما يبرهن هندسياً أن التوزيع يقلل الضغط عن الخادم الواحد ويزيد من توفر النظام (High Availability).

الطلب السادس: التخزين المؤقت الموزع (Distributed Caching)

كيف تم الحل:

تم اعتماد نمط Cache-Aside باستخدام Redis كطبقة تخزين مؤقت وسيطة بين التطبيق وقاعدة البيانات، لتخفيف الضغط الناتج عن قراءة المنتجات المتكررة. عند ورود طلب قراءة، يُفحص Redis أولاً؛ فإن وجدت البيانات تُرجع مباشرة (Cache HIT) دون لمس قاعدة البيانات، وإن لم توجد (Cache MISS) تُجلب من قاعدة البيانات وتُخزن في Redis لمدة محددة. تم التطبيق في موضعين:

- الموضع الأول — عند عرض المنتجات (ProductController):

```
$products = Cache::remember("products:page:{$page}", now()->addMinutes(5), function ()
{
    return Product::query()->latest()->paginate(20);});
```

استُخدم مفتاح منفصل لكل صفحة (products:page:N) بمدة صلاحية 5 دقائق، ومفتاح لكل منتج فردي (product:{id}) بمدة 10 دقائق. أول طلب يستعلم من قاعدة البيانات ويخزن النتيجة في Redis، وكل طلب تالي يُرجع من Redis مباشرة دون لمس قاعدة البيانات.

● الموضوع الثاني — إبطال الكاش بعد الشراء (CheckoutService):

```
Cache::deleteMultiple(
    $cartItems->pluck('product_id')->map(fn($id) => "product:{$id}")->all());
```

بعد نجاح المعاملة، يُحذف كاش المنتجات المشتراة دفعةً واحدة (batch) عبر deleteMultiple بدلاً من حلقة تُرسل N أوامر منفصلة لـ Redis، لضمان أن المستخدم التالي يرى مخزوناً محدثاً.

لماذا اخترنا هذا الحل:

اعتمدنا نمط Cache-Aside لأنه يفصل منطق التخزين المؤقت عن منطق العمل، ويبقي قاعدة البيانات مصدر الحقيقة. كما اتخذنا قراراً هندسياً مهماً: في نسخة أولية أضفنا جدول product_statistics لتتبع «المنتجات الأكثر طلباً» صراحةً، لكن القياس أظهر أنه يضيف استعلامي قراءة وكتابة على كل Cache MISS — مما يناقض هدف المتطلب نفسه (تقليل استعلامات قاعدة البيانات). لذا حذفناه بالكامل، وتحقق هدف «الأكثر طلباً» ضمناً: المنتج المطلوب بكثرة يبقى دافئاً في الكاش لتكرار طلبه قبل انتهاء مدة الصلاحية، والمنتج النادر يسقط تلقائياً. وللتحكم في الذاكرة عند النمو، اعتمدنا سياسة allkeys-lfu في إعداد Redis (طرد الأقل استخداماً) بدلاً من منطق إضافي في الكود.

أما قائمة المنتجات فلا تُبطل بشكل نشط عند الشراء، بل تعتمد على مدة الصلاحية القصيرة فقط؛ لأن قرار البيع الحقيقي يحدث في عملية الـ checkout على قاعدة البيانات عبر الخصم الذري المشروط (الطلب الأول)، فحتى لو عرضت القائمة مخزوناً قديماً لثوانٍ، لا يحدث بيع زائد (Overselling).

كيف تم التحقق:

الدليل الأساسي — تقليل استعلامات قاعدة البيانات: عبر السكريبت cache_benchmark_demo.php، انخفض عدد الاستعلامات المباشرة على قاعدة البيانات من 150 استعلاماً إلى 7 فقط عند تكرار الطلبات نفسها — أي بنسبة تخفيض 95%. هذا الدليل هو الأقوى لأنه يُقاس من داخل التطبيق مباشرة دون تأثر بضوضاء الشبكة.

الدليل المساند — زمن الاستجابة: عبر اختبار k6_cache_benchmark.js بعشرة مستخدمين متزامنين، قورن زمن الاستجابة (p95)

- قبل تفعيل الكاش وبعده: قبل تفعيل الكاش: بلغ زمن استجابة قائمة المنتجات p95 حوالي 6.32 ثانية، وزمن المنتج الفردي p95 حوالي 5.73 ثانية، مع معدل نجاح 100%.

- بعد تفعيل الكاش: انخفض زمن قائمة المنتجات p95 إلى 4.96 ثانية (تحسّن بنسبة 21%)، وزمن المنتج الفردي p95 إلى 4.68 ثانية (تحسّن بنسبة 18%)، مع بقاء معدل النجاح عند 100%.

- ملاحظة منهجية صادقة: التحسن الزمني محدود نسبياً لأن عملية تحويل البيانات إلى JSON ((Serialization تحدث في الحالتين، فالكاش يوفّر زمن استعلام قاعدة البيانات وتهيئة كائنات Eloquent فقط لا زمن التسلسل؛ لذلك يبقى عدد الاستعلامات (تخفيض 95%) هو الدليل القاطع على نجاح التخزين المؤقت لا الزمن وحده.

ملاحظة منهجية صادقة: التحسن الزمني محدود نسبياً لأن عملية تحويل البيانات إلى JSON Serialization تحدث في الحالتين — فالكاش يوفّر زمن استعلام قاعدة البيانات وتهيئة كائنات Eloquent فقط، لا زمن التسلسل. لذلك يبقى عدد الاستعلامات (تخفيض 95%) هو الدليل القاطع على نجاح التخزين المؤقت، لا الزمن وحده.

الطلب السابع: القفل الموزّع عبر Redis (Distributed Lock)

كيف تم الحل:

لمنع شراء نفس المنتج من مستخدمين في اللحظة نفسها قبل إتمام التحقق، تم اعتماد قفل موزّع (Distributed Lock) على Redis عبر Cache::lock — وهو قفل يعمل عبر الخوادم المتعددة (يخدم الطلب الخامس)، بخلاف قفل صفوف قاعدة البيانات المحصور بخادم واحد. يتم جمع معرفات المنتجات وترتيبها تصاعدياً ثم اكتساب القفل لكل منتج:

```
;()productIds = $cartItems->pluck('product_id')->unique()->sort()->values$

foreach ($productIds as $lockProductId) {
    $lock = Cache::lock("lock:product:{$lockProductId}", 10); // TTL 10s
    $lock->block(1); // انتظار قصير للحصول على القفل
    $locks[] = $lock;
}
```

ثلاثة تفاصيل تصميمية مهمة:

- **ترتيب المعرفات تصاعدياً عبر sort():** يمنع الجمود (Deadlock)؛ فلو اكتسب الطلب A قفل المنتجين {1, 2} بينما اكتسب الطلب B القفل بترتيب معكوس {2, 1}، لانتظر كل منهما الآخر إلى الأبد (توقف تام).
- والترتيب الثابت يضمن أن جميع الطلبات تكتسب الأقفال بالترتيب نفسه، فلا يحدث جمود.
- **مدة صلاحية 10 ثوانٍ على القفل (TTL):** تعمل كشبكة أمان؛ فلو انهارت العملية فجأة (crash) قبل تحرير القفل، يُحرّر تلقائياً بعد 10 ثوانٍ، فلا يبقى المنتج مقفولاً إلى الأبد.
- **التحرير داخل كتلة finally:** يضمن تحرير القفل دائماً حتى لو رُمي استثناء داخل المعاملة.

منطق إعادة المحاولة (Retry) مع التراجع الآسي:

```
if ($attempt < 3) {
    $backoffMs = 50 * pow(2, $attempt); // 50,100,200 ms
    usleep($backoffMs * 1000 + random_int(0, 30_000)); // jitter
    return $this->checkout($user, $attempt + 1);
}throw $exception;
```

عند ازدحام القفل، يرمي block استثناء LockTimeoutException فيعيد النظام المحاولة حتى ثلاث مرات مع تراجع أسّي وعشوائية (jitter) لتفادي تزامن المحاولات (Thundering Herd). إن استنفدت المحاولات يُرجع 503 (تخفيف حمل مهذب) بدل خطأ 500.

لماذا اخترنا هذا الحل:

القفل الموزع على Redis يلبي اشتراط استخدام قفل خارج قاعدة البيانات يعمل عبر عدة خوادم. والأهم أن القفل هنا أداة تنسيق (ينظم ترتيب الوصول) لا مصدر قرار؛ فالقرار النهائي حول توفر المخزون يبقى للخصم الذري على قاعدة البيانات (الطلب الأول). لذلك لا يسبب الكاش القديم رفضاً خاطئاً — القفل ينظم فقط، وقاعدة البيانات تحكم. كما ضُبط زمن الانتظار وعدد المحاولات بناءً على قياس فعلي أظهر أن متوسط زمن اكتساب القفل لا يتجاوز عشرات الملي ثانية، فاختير انتظار قصير (1) block ومحاولات قليلة لتحقيق التوازن بين النجاح وعدم احتجاز خيوط الخادم طويلاً.

الطلب الثامن: سلامة المعاملات (ACID Transaction)

كيف تم الحل:

لضمان أن جميع خطوات عملية الشراء تتجح معاً أو تفشل معاً دون أي حالة وسطية فاسدة، تم تغليف العملية كاملةً داخل معاملة واحدة DB::transaction تُنفَّذ بينما الأقفال الموزعة (الطلب السابع) محتجزة:

```
$order = DB::transaction(function () use (...) {
    Product::where('id', ...)->where('stock', '>=', ...)->decrement('stock', ...);

    $order = Order::create([...]);

    $order->items()->createMany($orderItemsPayload);

    $user->cartItems()->delete();

    return $order;
});
```

ترتيب التنفيذ داخل القفل:

اكتساب القفل ← بدء المعاملة ← كل العمليات ← إنهاء المعاملة ← تحرير القفل

لو فشلت أي خطوة داخل المعاملة (مثلاً المخزون غير كافٍ) يحدث تراجع تلقائي كامل (Rollback) لكل التغييرات — فلا بيع زائد ولا بيانات جزئية.

لماذا اخترنا هذا الحل:

تحقق المعاملة خصائص ACID الأربع بوضوح:

- (Atomicity) أي استثناء داخل المعاملة يُسبب تراجعاً كاملاً لجميع التغييرات.
- (Consistency) في الخصم الذري يمنع نزول المخزون تحت الصفر $stock \geq qty$ شرط.
- (Isolation) الخصم الذري المشروط يمنع تداخل المعاملات المتزامنة على الصف نفسه، يعزّزه القفل الموزع.
- (Durability) تبقى البيانات محفوظة — أثبت تجريبياً ببقائها بعد انهيار فعلي لـ (Commit) بعد التثبيت MySQL.

نقطة مهمة: المعاملة وحدها لا تمنع البيع الزائد — هي تضمن الذرية فقط؛ أما الذي يمنع البيع الزائد فعلياً فهو الخصم الذري المشروط (الطلب الأول). فالأداتان تعملان معاً: المعاملة للسلامة، والخصم الذري للصحة، والقفل الموزع للتنسيق — ثلاث طبقات لثلاثة أغراض مختلفة.

الطلب التاسع: اختبار الضغط (Stress Testing)

كيف تم الحل:

لقياس سلوك النظام تحت مئة مستخدم متزامن، بُني سكربت اختبار ضغط عبر أداة (k6 stress_test.js) (k6) يحاكي رحلة مستخدم كاملة، ويصنّف الردود تصنيفاً صادقاً (2xx ناجح، 422 نفاذ مخزون، 503 ازدحام، 5xx انهيار). يتدرّج الحمل عبر ثلاث مراحل:

رفع تدريجي من 0 إلى 100 مستخدم : 20s → 0s
ثبات على 100 مستخدم : 80s → 20s
هبوط تدريجي إلى 0 : 90s → 80s

رحلة كل مستخدم في كل دورة:

(الطلب 6) Redis قراءة مخزّنة في ← GET /api/products
إضافة للسلة ← POST /api/cart/add
القلب: قفل + معاملة + خصم ذري ← POST /api/checkout
عرض الطلبات ← GET /api/orders

صُمّمت بيانات الاختبار بثلاثة أصناف من المنتجات لمحاكاة سيناريوهات واقعية متنوعة:

- وفير (Abundant): 200 منتج بمخزون 100,000 — لا ينفذ أبداً، فيُستخدم لقياس خط الأساس.
- نادر (Scarce): 50 منتجاً بمخزون 20 — ينفذ في منتصف الاختبار فينتج ردود 422.
- ساخن (Hot): 10 منتجات بمخزون 5 — ينفذ سريعاً لقياس تنافس الأقفال.

كيف تم التحقق:

معيّار النجاح الجوهرى: عدم وجود أي انهيار (5xx = 0)، إذ تُعدّ ردود 422 (نفاذ مخزون) و503 (ازدحام) سلوكاً صحيحاً لا فشلاً. وأظهرت النتيجة النهائية تحت مئة مستخدم متزامن ما يلي:

- إجمالي الدورات المكتملة: 3457 دورة.
- إجمالي الردود: 13928 رداً.
- ناجحة (2xx): 11169.
- نفاذ مخزون (422) — سلوك صحيح: 2025.
- ازدحام (503) — تخفيف حمل: 734.
- انهيار (5xx): 0.
- زمن الشراء 4.08: p(95) ث، وأقصى زمن للشراء 4.79 ث.
- لا انهيار (NO CRASH): نعم.

فحص سلامة البيانات (الدليل على صحة الاختبار وعدم حدوث بيع زائد): بعد الاختبار تحقّقنا من قاعدة البيانات فلم يوجد مخزون سالب، وكان عدد الطلبات قبل الاختبار صفراً، وبعده صار عدد الطلبات مطابقاً لعدد عمليات التشيك-أوت

الناجحة التي تُطبع في الاختبار، ولا بيع زائد (النقص في المخزون يساوي تماماً الكمية المباعة) — مما يثبت أن آليات الطلبات الأولى والسابع والثامن صمدت تحت التزامن الكامل.

الطلب العاشر: تحليل الاختناقات (Bottleneck Analysis)

كيف تم الحل:

لتحديد موضع الاختناق بدقة بدلاً من التخمين، أُضيفت ثلاث طبقات قياس (Instrumentation) تقيس زمن كل مرحلة داخل الطلب الواحد دون تغيير أي منطق:

الطبقة الأولى — قياس زمن المعالجة الكلي في PHP Middleware:

```
$start = microtime(true);  
$response = $next($request);  
$response->headers->set('X-Process-Time-Ms', round((microtime(true)-$start)*1000,  
1));
```

يُميّز هذا بين أمرين: هل التأخير داخل معالجة PHP، أم في انتظار قبول الطلب من خادم Apache؟

الطبقة الثانية — إحصاء استعلامات قاعدة البيانات (Middleware):

```
X-Q-Count      → (N+1) عدد الاستعلامات (كاشف)  
X-Q-Total-Ms   → SQL مجموع زمن  
X-Q-Max-Ms     → أبطأ استعلام مفرد
```

الطبقة الثالثة — توقيت كل مرحلة داخل عملية الشراء (CheckoutService):

يحمل ترويسة X-Checkout-Profile قيم زمن كل مرحلة، يقرأها k6 ويبنى منها تفكيك المراحل التالي (متوسط الزمن ونسبته من زمن المعالجة):

- اكتساب القفل (lock)
- تحميل السلة (cart_load)
- تحميل الطلب (order_load)
- تكلفة التثبيت (commit)
- إنشاء البنود (items_create)
- الخصم الذري (decrement)
- إنشاء الطلب (order_create)
- تفريغ السلة (cart_delete)
- إرسال المهمة (dispatch)
- إبطال الكاش (cache_forget)

وقد حدّدنا عدة عقد اختناق وعالجنا كلاً منها بأداة مستقلة، مع توثيق رقمي قبل/بعد لكل عقدة:

عقدة تسرّب الذاكرة: حدّدت عبر مراقب الموارد الذي أظهر تصاعد استهلاك الذاكرة من 66 ميغابايت حتى 2,115 ميغابايت بشكل غير محدود، وكان سببها اعتماد محرّك الطوابير على قاعدة البيانات (QUEUE_CONNECTION=database)، إذ تتراكم المهام في جدول MySQL فتتفخ ذاكرته. عُولجت

بتحويل الطوابير إلى Redis لتُخزَّن المهام في الذاكرة الخفيفة بدل جدول قاعدة البيانات، فهبطت ذروة الذاكرة من 2,115 ميغابايت إلى 475 ميغابايت (بنسبة تخفيض 78%).

عقدة بطء قراءة المنتجات: حُدِّت عبر قياس زمن استجابة المسار GET /api/products الذي بلغ 95p فيه نحو 11,283 ميلي ثانية، وكان سببها تنفيذ استعلام كامل على قاعدة البيانات وإرجاع آلاف السجلات دفعةً واحدة في كل طلب. عُولِجَت بالتخزين المؤقت في Redis مع ترقيم الصفحات (20 منتجاً لكل صفحة)، فانخفض الزمن من 11,283 ميلي ثانية إلى 274 ميلي ثانية (بنسبة تحسَّن 97%).

عقدة بطء عرض الطلبات (مشكلة N+1): حُدِّت عبر قياس زمن المسار GET /api/orders الذي بلغ 95p فيه نحو 21,374 ميلي ثانية، إضافةً إلى عدَّاد الاستعلامات الذي كشف عدداً كبيراً منها، وكان سببها تحميل الطلبات والبنود والمنتجات بشكل منفصل (استعلام لكل عنصر = N+1). عُولِجَت بالتحميل المسبق (Eager Loading) عبر with('items.product') لتُجلب البيانات في استعلامات قليلة مُجمَّعة، فانخفض الزمن من 21,374 ميلي ثانية إلى 302 ميلي ثانية (بنسبة تحسَّن 99%).

عقدة بطء تحليل PHP في كل طلب: حُدِّت عبر طبقة قياس زمن المعالجة التي أظهرت عبئاً ثابتاً على كل الطلبات، وكان سببها أن PHP بدون OPcache يُعيد قراءة وترجمة مئات ملفات Laravel من القرص في كل طلب. عُولِجَت بتنفيذ OPcache مع تجميع إعدادات ومسارات Laravel مسبقاً (config:cache و route:cache و event:cache)، فبقي الكود المترجم في الذاكرة المشتركة ولم تعد الملفات تُترجم في كل طلب.

عقدة ضبط محرك قاعدة البيانات InnoDB: حُدِّت عبر طبقة قياس زمن استعلامات SQL وسلوك الكتابة تحت الحمل، وعُولِجَت بأربعة إعدادات أداء لا تمس سلامة البيانات: innodb_buffer_pool_size بقيمة 256 ميغابايت ليخزَّن صفحات البيانات في الذاكرة بدل قراءتها من القرص في كل استعلام، و innodb_log_file_size بقيمة 48 ميغابايت ليقَلَّ تكرار نقاط التفتيش وذبذبات الإدخال/الإخراج، و innodb_thread_concurrency بقيمة 8 ليحدَّ عدد خيوط InnoDB المتزامنة ويقلِّل تنافس المعالج، و skip_name_resolve لإلغاء استعلام DNS عند كل اتصال بقاعدة البيانات.

ولتأكيد دقة التشخيص أجرينا تجربتي نفي: الأولى رفع عدد خيوط Apache، ولم تُحدث أي فرق لأن الخيوط ليست العنق (عددها أصلاً أكبر من عدد المستخدمين والمعالج غير مشبع). والثانية التوزيع على خادمين على الجهاز نفسه، وقد ساء فيها الأداء نحو 2.5 ضعفاً لأن الخادم الإضافي كان أحادي الخيط ويتقاسم العتاد نفسه، مما يثبت أن التوسّع الأفقي لا يفيد إلا بخوادم متكافئة على عتاد منفصل.
